

Pareto Crewing

Decentralised, privacy-preserving measurement of language-model teams,
and a learner that picks them on the cost-quality front, per role and per task

Santosh Kumar Radha

AgentField AI

September 2026

Abstract

Pareto crewing is the practice of picking the language-model team for a task on the cost-quality front, one model per role, from evidence about how models have performed in that role rather than from a fixed table. An agent harness runs a task through a *crew*: an assignment of one language model to each of n roles (in our instance a worker that does the work, a checker that reads it and a planner that cuts it). The crew that pays best is neither the cheapest nor the strongest on paper; it is the one that minimises the expected cost of one clean delivery once failed runs, wall time and shipped defects are priced. We give that objective in closed form, show how a crew is computed from a published catalog of prices and benchmark indexes alone (a pareto front in bill and quality, with a scale-free knee), and then how measurements from real runs replace the catalog’s proxy. Runs are scored role by role by a model outside the crew. Each score becomes a row that carries no text, no file name, no price and no identity beyond a per-install nonce, so what an install sends is a handful of coarse fields about a model, not about the work; rows reduce to sufficient statistics that form a join-semilattice, so any number of installs and a relay can merge them in any order and reach the same state, the relay is a convenience rather than an authority, and the published index is signed so that every reader can check what it is blending in. Participation is a setting: an install can contribute, read and learn from its own runs without sending, or stay out. Judge severity is removed by a two-way fixed-effects fit on those statistics, drift by a local-level model whose forgetting rate is estimated rather than set, and heterogeneity between installs by a hierarchical prior whose variance is estimated by moments; every statistical constant is an empirical-Bayes estimate, and what remains for the user to set is the price of a defect and the price of time. The crew is chosen by Thompson sampling with semi-bandit feedback: because the objective is linear in role qualities and every role is scored separately, a run on one crew informs every crew that shares a model with it, and the sample complexity scales with the number of models rather than the number of crews. On a ground truth fitted to our own runs the learner beats a fixed table by a third of the remaining regret within a few hundred runs, and role-level feedback halves the regret of a crew-level learner fed the same scores at four hundred runs.

Keywords: Pareto crewing, model selection, pareto front, empirical Bayes, Thompson sampling, combinatorial semi-bandits, CRDT, judge calibration, privacy, decentralised measurement.

1 Introduction

A harness that drives language models through software tasks has to decide, before every task, which model sits in which role. The decision is made under three kinds of information that arrive at different times and in different units: published prices and benchmark indexes, available for every model on the day it appears; the harness’s own past runs, few but on the right distribution; and the runs of everyone else using the same harness, many but on other people’s tasks and scored by other people’s judges. This note is about turning the three into one decision.

We call the resulting discipline *Pareto crewing*: the crew is picked on the cost-quality front, per role and per task, from whatever evidence exists on the day, and the evidence is pooled across installs without any install giving up what it worked on. Our contributions are the following.

1. A closed-form objective for a crew, the expected cost of one clean delivery under reruns and a priced defect (Section 4), and the observation that on a catalog alone the objective cannot be evaluated, which is why the catalog picker is a pareto front with a scale-free knee (Section 3).

2. A decentralised measurement of role quality (Section 5): text-free rows, sufficient statistics that form a state-based CRDT (Proposition 2), judge fixed effects identified from the connectivity of the judge-cell graph (Proposition 3), robust aggregation across installs, and drift handled by a local-level model.
3. An estimator with no tuned constant except the user’s price of a defect (Section 6): the catalog’s prior weight, the between-install variance, the noise scale and the forgetting rate are all estimated from the pool by moments.
4. A picker (Section 7) that is Thompson sampling in a combinatorial semi-bandit: the crew objective is linear in role qualities, roles are scored separately, so one run updates every crew sharing a model with it.
5. An empirical check on our own measured runs (Section 8), reproducible from the script beside this document.

2 Setting

Definition 1 (Roles, crews). Let $\mathcal{R}$ be a finite set of n roles and $\mathcal{M}$ a finite set of models. A *crew* is a map $\kappa : \mathcal{R} \rightarrow \mathcal{M}$. Each role r carries a *shape*: a token share $v_r \geq 0$ with $\sum_r v_r = 1$, index weights $w_r \in \Delta^2$, a cache share $\gamma_r \in [0, 1]$, an input-to-output ratio $\rho_r > 0$, and a predicate $\text{fit}_r(m)$ saying whether model m can hold the role (tools, images, context). A symmetric relation $D \subseteq \mathcal{R} \times \mathcal{R}$ names the pairs of roles that may not be held by models of the same vendor.

Our instance has $\mathcal{R} = \{\text{W}, \text{H}, \text{M}\}$ (worker, high, mastermind), token shares $(0.90, 0.08, 0.02)$, weights $(0.2, 0.3, 0.5)$, $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$, $(0.6, 0, 0.4)$ on the indexes (intelligence, coding, agentic), cache shares $(0.75, 0.75, 0)$, ratios $(100, 100, 10)$ and $D = \{(\text{W}, \text{H})\}$: the two roles that read the same work never share one shop. On the chat door the planner role is absent and $n = 2$. Nothing below depends on $n = 3$.

A *task* is one unit of work; a *run* is one attempt at it by a crew. A run ends *delivered* or not, and a delivered task is later found *clean* (no major defect) or not. Money is in dollars and time is priced at λ dollars per minute.

The *catalog* lists, per model, a prompt price π_m , a cache-read price π_m^c , a completion price κ_m per million tokens, and benchmark indexes $I_m \in \mathbb{R}^3$. The *pool* is the set of measurements described in Section 5; the *own sheet* is one install’s part of it.

3 The crew computed from the catalog

Role cost and role quality. Per million task tokens on role r , model m costs

$$c_r(m) = (1 - \gamma_r) \pi_m + \gamma_r \pi_m^c + \kappa_m / \rho_r, \quad (1)$$

and its catalog quality is the weighted, pool-normalised index

$$\hat{q}_r(m) = 100 \sum_{i=1}^3 w_{r,i} \frac{I_{m,i}}{\max_{m' \in \mathcal{M}} I_{m',i}}. \quad (2)$$

A missing index is imputed from the models that publish it: for each index the model does publish, the median over at least three donor models of the ratio between the missing and the present index gives an offer, the offers are averaged, and the estimate is capped at the largest measured value of the missing index in the pool; with fewer than three donors the model takes the standing its present indexes give it on the pool’s scale. An imputed quality is flagged and loses ties.

Shortlists. Role r ’s shortlist is

$$S_r = \{m : \text{fit}_r(m) \wedge \hat{q}_r(m) \geq \phi \max_{m'} \hat{q}_r(m')\}, \quad \phi = 0.8, \quad (3)$$

the floor being the statement that a model which cannot do the job is not cheap at any price. The crew set is $\mathcal{K} = \{\kappa \in \prod_r S_r : \text{vendor}(\kappa(r)) \neq \text{vendor}(\kappa(r')) \forall (r, r') \in D\}$.

Bill, quality, front.

$$B(\kappa) = \sum_r v_r c_r(\kappa(r)), \quad Q(\kappa) = \sum_r v_r q_r(\kappa(r)), \quad v_r \geq 0, \sum_r v_r = 1, \quad (4)$$

with v uniform by default. The *front* $\mathcal{F} \subseteq \mathcal{K}$ is the set of crews not dominated in (B, Q) . Sorting $\mathcal{K}$ by $(B \uparrow, Q \downarrow, \text{imputed count } \uparrow, \text{id } \uparrow)$ and keeping a crew when its Q exceeds every kept crew's yields $\mathcal{F}$ in $O(|\mathcal{K}| \log |\mathcal{K}|)$, and the tie order makes $\mathcal{F}$ a function of the set $\mathcal{K}$ rather than of any input order. In practice $|\mathcal{S}_r|$ is a few dozen and $|\mathcal{K}|$ a few thousand.

Why a front and not a score. The objective of Section 4 needs a probability of clean delivery. The catalog has none, so any scalarisation of (B, Q) would fix an exchange rate between a price and a benchmark index that nobody has measured. Dominance is the only order that needs no such rate; every crew a user could sensibly want is on $\mathcal{F}$, which is small, and the remaining choice is one-dimensional along it.

Knee and knob. Index $\mathcal{F}$ by bill, $B_0 \leq \dots \leq B_1$. The three named picks are *frugal* $= \arg \min_{\mathcal{F}} B$, *max* $= \arg \max_{\mathcal{F}} Q$, and *balanced*, the crew farthest above the chord of the front in $(\ln B, Q)$:

$$\text{balanced} = \arg \max_{\kappa \in \mathcal{F}} \left[(Q(\kappa) - Q_0) - (Q_1 - Q_0) \frac{\ln B(\kappa) - \ln B_0}{\ln B_1 - \ln B_0} \right]. \quad (5)$$

Proposition 1 (Scale invariance). *The balanced crew is invariant under $B \mapsto aB$ for any $a > 0$ and under $Q \mapsto Q + b$.*

Proof. $\ln(aB) - \ln(aB_0) = \ln B - \ln B_0$ and $\ln(aB_1) - \ln(aB_0) = \ln B_1 - \ln B_0$; the bracket is unchanged. A shift of Q cancels in both differences. $\square$

The log scale is not cosmetic: catalog prices span two orders of magnitude, and the knee should not move when a currency changes or every vendor cuts prices by the same factor. A knob $k \in [0, 1]$ picks the best crew with $B \leq B_0(B_1/B_0)^k$, so equal steps in k are equal ratios in the bill.

4 The objective: the cost of one clean delivery

Let a run of crew κ cost c dollars and t minutes, deliver with probability r , and let a delivered task be clean with probability p ; a run that fails to deliver costs c_f and t_f .

Lemma 1 (Rerun until clean). *If every defect is caught and the task rerun, the expected cost to one clean delivery is*

$$E_0(\kappa) = \frac{(1-r)(c_f + \lambda t_f) + r(c + \lambda t)}{rp}. \quad (6)$$

Proof. Attempts are i.i.d. with success probability rp ; the count to the first success is geometric with mean $1/(rp)$, and by Wald's identity the expected total cost is the mean count times the expected cost of one attempt, which is the numerator. $\square$

Lemma 2 (Priced defect). *If a shipped defect costs H to find and mend and only non-deliveries are rerun,*

$$E_H(\kappa) = \frac{(1-r)(c_f + \lambda t_f)}{r} + (c + \lambda t) + (1-p)H. \quad (7)$$

Proof. The count of failed attempts before the first delivery is geometric on $\{0, 1, \dots\}$ with mean $(1-r)/r$; the delivering attempt costs $c + \lambda t$; the defect is paid once with probability $1-p$. $\square$

H is the one quantity that belongs to the user, and the budget word sets it: frugal $H = \$1$, balanced $\$10$, max $\$50$. Both forms are monotone in the same three quantities, the run's cost, its time and $P_c = rp$, and the picker works with

$$E(\kappa) = c(\kappa) + \lambda t(\kappa) + H(1 - P_c(\kappa)). \quad (8)$$

Equation (8) is (7) with r set to one: a run that fails to deliver is priced as a shipped defect rather than rerun. The surrogate ranks crews as E_H does when H is large next to a run's cost and r is not far below one, which is the regime of every crew on the front of Figure 1; when a role's delivered rate is measured, as the worker's is, the picker can use (7) directly and nothing below changes. Figure 1 draws our measured crews with the iso-lines of (8). The knob k and the price H parametrise the same trade-off; k is what a catalog admits, H is what measurements admit.

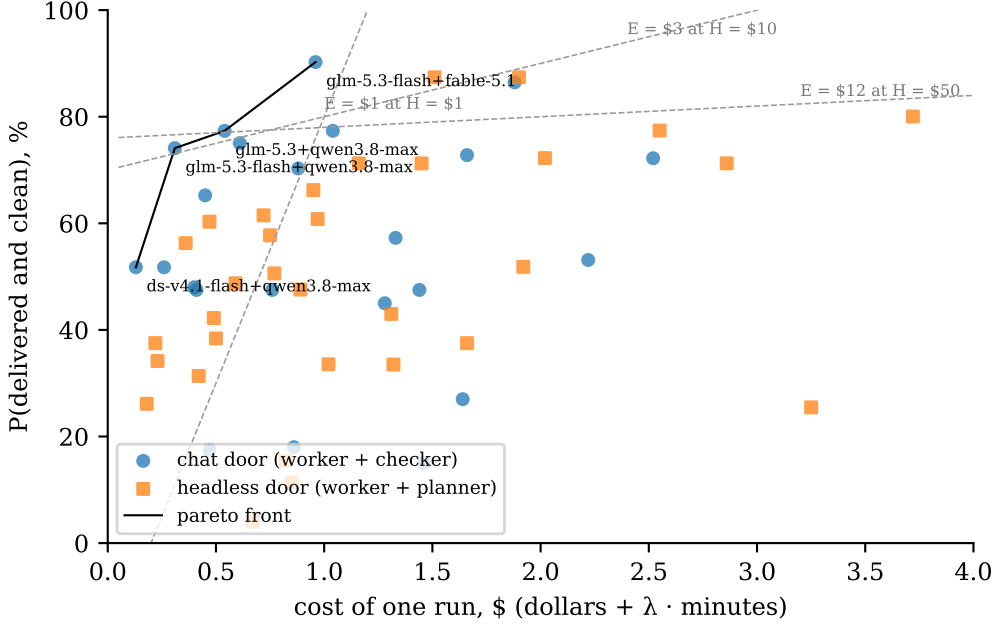


Figure 1: Our measured crews: 56 (worker, checker or planner) combinations, each fitted by a ridge model on 72 scored runs over nine tasks; cost of one run against the probability that it delivers clean. Solid: the pareto front. Dashed: iso-cost lines of (8); a larger H tilts them so that quality buys more.

Assumption 1 (Linearity in roles). $P_c(\kappa) = \sum_r \nu_r q_r(\kappa(r))/100$, where $q_r(m)$ is the pool’s role quality: the run’s outcome credited to the role, on a 0–100 scale.

The assumption is what makes (8) linear in the role qualities, which Section 7 uses. It is a working rule rather than a law: a product over roles would count one outcome n times, and a fitted interaction becomes available once judged scores separate the roles’ contributions. Nothing in Sections 5–6 depends on it.

5 The Model Pool: a decentralised measure of role quality

5.1 Judged runs and rows

After a run finishes, one model outside the crew reads the record (brief, deliverable, the checker’s report and claim, the files touched) and answers one question per role with a score in $[0, 100]$ and one sentence. The judge is chosen deterministically: the cheapest catalog model with coding index at least 60 that can call tools, holds no role in the crew, and is not from the worker’s vendor; a role the judge’s own model holds is skipped. The floor is absolute, not relative to the worker; its reason is that a judge well below the class of the work it reads rubber-stamps. The judge’s calls are billed to a role of their own, so learning is never free by accident; the cost is one short completion per role.

One judged role of one run is one *row*

$$x = (\text{role}, \text{model}, \text{score}, \text{judge}, \text{door}, \text{size} \in \{S, M, L\}, \text{day}),$$

sent under an *install* nonce drawn once per profile and carried on the batch, not in the row. No text, no price and no duration is in a row; dollars and minutes stay in the install’s own ledger. Rows wait in a local outbox, which can be inspected, until they are sent.

5.2 Sufficient statistics and order-free merging

Definition 2 (Cell, sheet). A *cell* is a tuple $\theta = (\text{role}, \text{model}, \text{judge}, \text{door}, \text{size})$. A *sheet* is a map from (install, day, cell) to a triple $T = (n, S, S^2) \in \mathbb{N} \times \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0}$, the count, sum and sum of squares of the scores an install has produced in that cell.

Every quantity computed below is a function of the sheet. Sheets are merged by the join

$$(A \sqcup B)(u, d, \theta) = (\max(n_A, n_B), \max(S_A, S_B), \max(S_A^2, S_B^2)) \quad \text{componentwise on } A(u, d, \theta), B(u, d, \theta), \quad (9)$$

with a missing entry read as $(0, 0, 0)$.

Proposition 2 (The sheet is a state-based CRDT). *Let each install u only ever publish its own entries (u, d, θ) and let every published triple be the running total of u 's rows in θ on day d . Then (i) $\sqcup$ is commutative, associative and idempotent; (ii) an install's triples are monotone non-decreasing in time, so a later publication dominates an earlier one under $\sqcup$; (iii) any two nodes that have received the same set of publications, in any order and with any duplication, hold the same sheet.*

Proof. (i) is the componentwise max on a product of totally ordered sets. (ii) Scores lie in $[0, 100]$, so n , S and S^2 each grow by a non-negative amount per row. (iii) follows from (i) and (ii): the join of a set of publications is their least upper bound, which does not depend on order or multiplicity, and by (ii) the least upper bound of an install's publications is its latest. $\square$

The cell means and pooled variances are functions of $\sum_u T(u, \theta)$, so by Proposition 2 a relay and every install compute the same estimates from the same publications; there is no coordinator whose order of arrival matters, and a relay that is lost is rebuilt from any node that kept the publications. This is the sense in which the measure is decentralised: the relay is a convenience for bandwidth and signing, not a source of truth. Two stores exist. The relay's store is the sheet of Definition 2, keyed by install and day and merged by (9). The install's own sheet (internal/pool/tally) is keyed by cell alone and merges by addition, which is safe there because an install folds each of its own rows in exactly once, as it is judged; it is the relay, which receives publications from many senders with retries, that needs the join.

5.3 Judges are not the same ruler

Let y be a score of role r , model m , by judge j . We posit the two-way additive model

$$y = \mu_{rm} + \beta_j + \varepsilon, \quad \mathbb{E}\varepsilon = 0, \quad \text{Var } \varepsilon = \sigma^2, \quad (10)$$

with μ_{rm} the quantity of interest and β_j the judge's severity, constrained by $\sum_j n_{..j}\beta_j = 0$ (the judges' weighted mean is the ruler). Weighted least squares needs only the cell means $\bar{y}_{rmj} = S_{rmj}/n_{rmj}$ and counts. The normal equations are solved by alternating

$$\beta_j \leftarrow \frac{\sum_{r,m} n_{rmj}(\bar{y}_{rmj} - \mu_{rm})}{\sum_{r,m} n_{rmj}}, \quad \mu_{rm} \leftarrow \frac{\sum_j n_{rmj}(\bar{y}_{rmj} - \beta_j)}{\sum_j n_{rmj}}, \quad (11)$$

re-centring β after each sweep. Each half-step minimises a convex quadratic in its block, so the residual is non-increasing and the iteration converges to the least-squares solution; five sweeps suffice in practice.

Proposition 3 (Identifiability). *Let G be the bipartite graph with the judges on one side, the pairs (r, m) on the other, and an edge where $n_{rmj} > 0$. The parameters of (10) are identified up to one additive constant per connected component of G : within a component every contrast is identified, and one centring constraint per component fixes its levels. When G is connected the single constraint identifies everything.*

Proof. The design matrix of a two-way additive model has rank equal to (number of vertices of G) minus (number of connected components), and its kernel is spanned by the vectors that shift μ up and β down by one constant on one component; one linear constraint per component removes them. With the single centring constraint, exactly one component is fully identified. $\square$

The relay centres β within each component. The judge rule of Section 5.1 makes G connected quickly in practice: it sends most installs to the same few cheap judges. A model scored by a single judge so far has μ_{rm} equal to its own mean and the index says so.

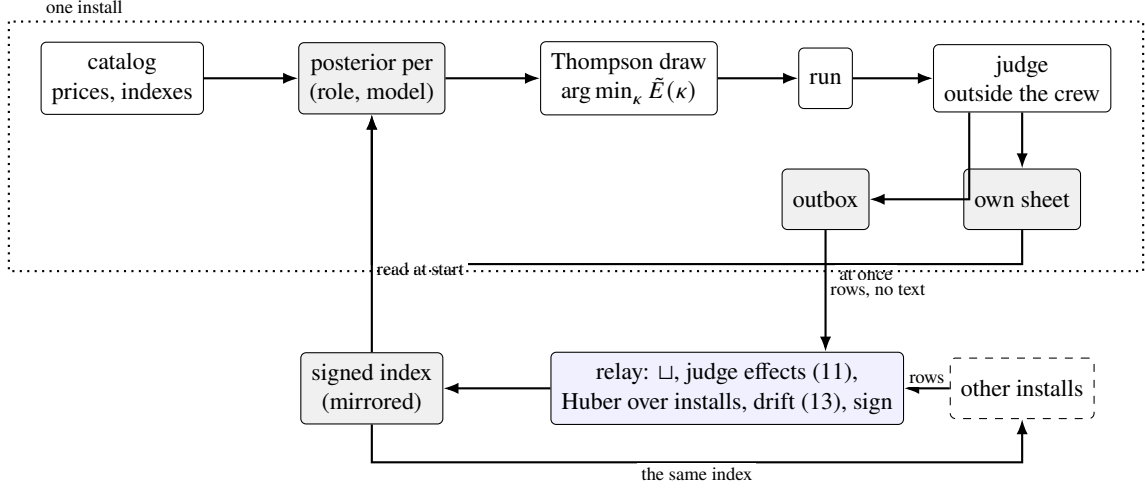


Figure 2: Where each step runs. Inside the dotted box is one install; the only thing that crosses it outward is a row without text. The relay merges publications with the join of Proposition 2 and publishes a signed index; any install can host a relay, and any node that kept the publications can rebuild one.

5.4 Installs are not independent, and some are wrong

Rows from one install share a task mix, a codebase and a way of writing briefs. Under an intra-class correlation ϱ , n_u rows from install u carry the information of $n_u / (1 + (n_u - 1)\varrho)$ independent ones, which saturates at $1/\varrho$. The pool’s cell estimate therefore treats installs, not rows, as the unit: it forms the install means $\bar{y}_{u\theta}$ and combines them with a location estimator that is robust to a minority of installs that are broken or adversarial. We use the Huber M -estimate with scale $s = 1.4826 \cdot \text{MAD}$ and the conventional tuning constant 1.345 (95% efficiency at the normal), which has breakdown point $1/2$ in location, and fall back to the precision-weighted mean when a cell has fewer than five installs. A cell backed by fewer than three installs is published but not read.

5.5 Drift

Models change under the same name. Let $\bar{y}_d$ be a cell’s day- d mean over n_d rows and model the true level as a random walk,

$$\mu_d = \mu_{d-1} + \eta_d, \quad \eta_d \sim \mathcal{N}(0, \omega^2); \quad \bar{y}_d = \mu_d + \bar{\varepsilon}_d, \quad \bar{\varepsilon}_d \sim \mathcal{N}(0, \sigma^2/n_d). \quad (12)$$

The Kalman filter for this local-level model is, in steady state with constant $n_d = \bar{n}$, an exponentially weighted average whose forgetting factor $1 - \alpha$ solves $\alpha^2 / (1 - \alpha) = \bar{n}\omega^2 / \sigma^2$; the effective memory is about $1/\alpha$ days. The drift variance is estimated by moments from the day-to-day differences,

$$\hat{\omega}^2 = \left[\widehat{\text{Var}}(\bar{y}_d - \bar{y}_{d-1}) - \sigma^2 \left(\frac{1}{n_d} + \frac{1}{n_{d-1}} \right) \right]_+, \quad (13)$$

pooled across cells. A cell with no evidence of drift ($\hat{\omega}^2 = 0$) keeps everything; a cell that drifts forgets at the rate the data show. The day is a cell dimension for exactly this reason.

6 Estimation with no tuned constant

We now combine the three sources for one install u . Write $\hat{q}_r(m)$ for the catalog quality (2), $\bar{y}_{rm}^{\text{pool}}$ and n_{rm}^{pool} for the pool’s cell after Sections 5.3–5.5, and $\bar{y}_{rm}^u$, n_{rm}^u for the own sheet.

Assumption 2 (Hierarchy).

$$\mu_{rm} \sim \mathcal{N}(\hat{q}_r(m) + b, \tau_0^2), \quad \mu_{rm}^u \mid \mu_{rm} \sim \mathcal{N}(\mu_{rm}, \tau_1^2), \quad y \mid \mu_{rm}^u \sim \mathcal{N}(\mu_{rm}^u, \sigma^2). \quad (14)$$

The first line says the catalog predicts the population quality up to one offset b shared by every model (the catalog’s scale is not the judge’s) and a noise τ_0 ; the second that install u ’s own quality for the model scatters around the population’s; the third is judge noise after Section 5.3. The install cares about μ_{rm}^u .

Proposition 4 (Posterior for the install). *Under (14), with $k = \sigma^2/\tau_0^2$, $\ell = \sigma^2/\tau_1^2$ and $\tilde{n} = n^{\text{pool}} \ell/(\ell + 1)$ (pool rows are other installs' rows, so each carries variance $\sigma^2 + \tau_1^2$ about μ_{rm}), the posterior of μ_{rm}^u is normal with*

$$\mathbb{E}[\mu_{rm}^u | \cdot] = \frac{k^* (\hat{q}_r(m) + b) + n^* \bar{y}_{rm}^{\text{pool}} + n^u \bar{y}_{rm}^u}{k^* + n^* + n^u}, \quad \text{Var}[\mu_{rm}^u | \cdot] = \frac{\sigma^2}{k^* + n^* + n^u}, \quad (15)$$

where the catalog and the pool share one capped external count W in proportion to what each contributed,

$$W = \frac{K \ell}{K + \ell} \leq \ell, \quad K = k + \tilde{n}, \quad k^* = \frac{k}{K} W, \quad n^* = \frac{\tilde{n}}{K} W. \quad (16)$$

Proof. Integrate out μ_{rm} . The catalog and the pool inform μ_{rm}^u only through μ_{rm} , whose posterior given them is normal with precision K/σ^2 , $K = k + \tilde{n}$, and mean the $k : \tilde{n}$ precision-weighted average of $\hat{q}_r(m) + b$ and $\bar{y}_{rm}^{\text{pool}}$; adding the between-install variance τ_1^2 gives, for μ_{rm}^u , a precision $1/(\sigma^2/K + \tau_1^2) = W/\sigma^2$ with the same mean. The own rows add precision n^u/σ^2 directly. Precision-weighting the two gives (15), with the external count W split as $k^* + n^* = W$ in the ratio $k : \tilde{n}$. $\square$

Corollary 1. *With $\tau_1 = 0$ ($\ell = \infty$) the posterior mean is $(1 - w)\hat{q}_r(m) + w\bar{y}$ with $w = n/(n + k)$, $n = n^{\text{pool}} + n^u$: the plain shrinkage blend, which is what the picker applies today with $k = 30$.*

The cap (16) is the point of the hierarchy. However many rows the pool holds, everything external to this install, catalog and pool together, is worth at most $\ell = \sigma^2/\tau_1^2$ own rows, because other installs' tasks are not this install's tasks; and the own rows are never capped. This replaces an ad hoc "own rows count triple" by a quantity with a meaning and an estimator. The pick shipped today is the Corollary's form: the pool's and the install's cells are folded together by observation count against the catalog at $k = 30$, which is (15) at $\ell = \infty$ and $b = 0$.

Estimating the constants. Every constant in (15) is estimated from the sheet by moments; none is set by hand.

$$\hat{\sigma}^2 = \frac{\sum_{u,\theta} (S_{u\theta}^2 - S_{u\theta}^2/n_{u\theta})}{\sum_{u,\theta} (n_{u\theta} - 1)} \quad \text{pooled within-cell variance,} \quad (17)$$

$$\hat{\tau}_1^2 = \left[\widehat{\text{Var}}_u(\bar{y}_{u\theta}) - \hat{\sigma}^2 (1/n_{u\theta}) \right]_+ \quad \text{between-install variance, pooled over cells,} \quad (18)$$

$$\hat{b} = \overline{\bar{y}_{rm}^{\text{pool}} - \hat{q}_r(m)} \quad \text{the catalog's offset,} \quad (19)$$

$$\hat{\tau}_0^2 = \left[\widehat{\text{Var}}_{(r,m)}(\bar{y}_{rm}^{\text{pool}} - \hat{q}_r(m)) - (\hat{\sigma}^2 + \hat{\tau}_1^2)/n_{rm}^{\text{pool}} \right]_+ \quad \text{how far the catalog misses,} \quad (20)$$

each a within-minus-noise moment estimator of the kind used by Efron and Morris [6]. Then $k = \hat{\sigma}^2/\hat{\tau}_0^2$ and $\ell = \hat{\sigma}^2/\hat{\tau}_1^2$, and $\hat{\omega}^2$ of (13) sets the forgetting. On the seed index alone the estimator already says something: its thirteen cells with a published intelligence index give $\hat{b} = -15.7$ (the catalog's scale sits sixteen points above the judged one) and a variance of the residuals, 75, below the noise they carry, 231, so $\hat{\tau}_0^2$ clips to zero: thirteen cells of bit-valued scores ($\hat{\sigma} \approx 44$) cannot resolve a catalog error under about nine points, and the day-one $k = 30$ stands until judged scores, with their smaller σ , replace the bits. These are published in the index beside the cells so that every install blends with the same constants, and re-estimated at every relay cycle. The relay withholds an estimate until its denominator has at least ten degrees of freedom and publishes the day-one values of Table 1 until then.

6.1 Covariates, and why no row is ever wasted

A row carries more than (role, model, score): a door, a size bucket, a judge, a day, an install, and any field added later, such as the harness version or a task family. Call the vector of such fields the row's *profile* $c \in C = \prod_f L_f$, a product of factor levels. The hierarchy (14) extends factor by factor:

$$\mu_{rm,c} = \mu_{rm} + \sum_f \delta_{rm,c,f}^f, \quad \delta_{rm,\cdot}^f \sim \mathcal{N}(0, \tau_f^2) \text{ i.i.d. over levels,} \quad (21)$$

so that a (role, model) has one population quality and one partially pooled offset per level of each factor. The install factor is the case $f = \text{install}$ of (21), and Proposition 4 is its two-level instance; the same derivation gives, for a query profile c , a posterior mean that is a precision-weighted combination of the cell means at every level that shares a coordinate with c , each capped by its own $\ell_f = \sigma^2/\tau_f^2$, with $\hat{\tau}_f^2$ estimated by the between-level moment (18) applied to factor f . Two properties follow.

Old rows keep their value when the harness changes. If the door algorithm is rewritten, rows from the old door are a level of the door factor; they still inform μ_{rm} with the full weight the estimated τ_{door}^2 allows, and the new door starts at μ_{rm} plus nothing and specialises as its own rows arrive. If the two doors turn out to differ little, $\hat{\tau}_{\text{door}}^2 \approx 0$, $\ell_{\text{door}} \rightarrow \infty$, and the factor pools completely; if they differ a lot, the cap keeps the old door from speaking for the new one. The data decide, per factor, how much they transfer.

The schema is open. A row with a field absent (older rows, before the field existed) is at a level of its own, *unknown*, which the estimator treats like any other level. Adding a field therefore invalidates nothing already collected, and the sheet’s join in Proposition 2 is unaffected because a cell key is only a tuple. What can be extracted from the rows is the set of contrasts the factors expose; a factor that is never recorded cannot be recovered later, which is why the row carries every cheap field now and the text of none.

Regressors that are not factors, a task’s token count for instance, enter the same way as a linear term with a partially pooled slope; the moment estimators are the same with a design column in place of an indicator.

Interactions: the crew as a group. Assumption 1 scores roles one at a time. Whether a particular worker and a particular checker do better *together* than their role qualities predict is a question about the pair, and the same hierarchy answers it: a pair $(m_r, m_{r'})$ is a level of a factor over $\mathcal{M} \times \mathcal{M}$, with its own partially pooled offset $\delta_{(m_r, m_{r'})}^{rr'} \sim \mathcal{N}(0, \tau_{rr'}^2)$. A pair that has never been run sits at zero offset and the additive prediction stands; a pair with rows moves by exactly the amount its rows justify against $\hat{\tau}_{rr'}^2$; if pairs in general add nothing, $\hat{\tau}_{rr'}^2 \rightarrow 0$ and the term disappears. This is the whole difference between learning crews as groups and learning roles: the group term is there, but it is charged for. A crew-level learner with no structure has to visit each of $|\mathcal{K}|$ crews to say anything about it; the additive model needs $\sum_r |S_r|$ observations to say something about every crew, and the pair term then spends further observations only where a pair is actually run. Some group effects are constraints rather than parameters: the vendor rule D is one, and it is applied to $\mathcal{K}$ before any learning happens.

constant	meaning	estimator	day-one value
σ	judge noise	(17)	15
$k = \sigma^2 / \tau_0^2$	weight of the catalog	(20)	30
$\ell_f = \sigma^2 / \tau_f^2$	cap on evidence transferred across a factor (install, door, size, ...)	(18) per factor	30
ω^2	drift per day	(13)	0 (no forgetting)
β_j	judge severity	(11)	0
v_r	role token shares	measured from run token ac- counting	0.90/0.08/0.02
H	price of a shipped defect	the user’s, by budget word	\$1 / \$10 / \$50
λ	price of a minute	the user’s	\$0.01
ϕ , judge floor	structural floors	fixed by design	0.8, 60

Table 1: Every constant with its estimator. Two are the user’s by design; two are structural floors. The rest are empirical-Bayes estimates that the relay publishes with the index.

7 Deciding: Thompson sampling with semi-bandit feedback

Given a posterior $\mathcal{N}(\mu'_{rm}, \sigma'^2_{rm})$ per role and model from (15), the pick could minimise (8) at the posterior means. That never tries a crew the prior underrates, and the catalog prior is wrong by a lot for some models (Figure 6). Thompson sampling [1, 2] draws one plausible world from the posterior and acts as if it were true: a crew is chosen with the probability that it is the best, exploration is proportional to what is unknown, and it stops on its own as the posteriors tighten. No exploration rate exists to be set.

Proposition 5 (Role draws are crew draws). *Under Assumption 1, drawing $\tilde{q}_{rm}$ independently per (role, model) and minimising $\tilde{E}$ over $\mathcal{K}$ is Thompson sampling over the crews’ posterior costs under the product posterior on role qualities.*

Proof. $\tilde{E}(\kappa)$ is an affine function of the vector $(\tilde{q}_{rm})$, so its joint law over $\mathcal{K}$ is the push-forward of the posterior on role qualities; a sample of the latter is a sample of the former. The judge-effect fit of Section 5.3 and the drift model of Section 5.5 induce dependence between cells that the independent draw ignores; the draw is exact for the product approximation the index publishes. $\square$

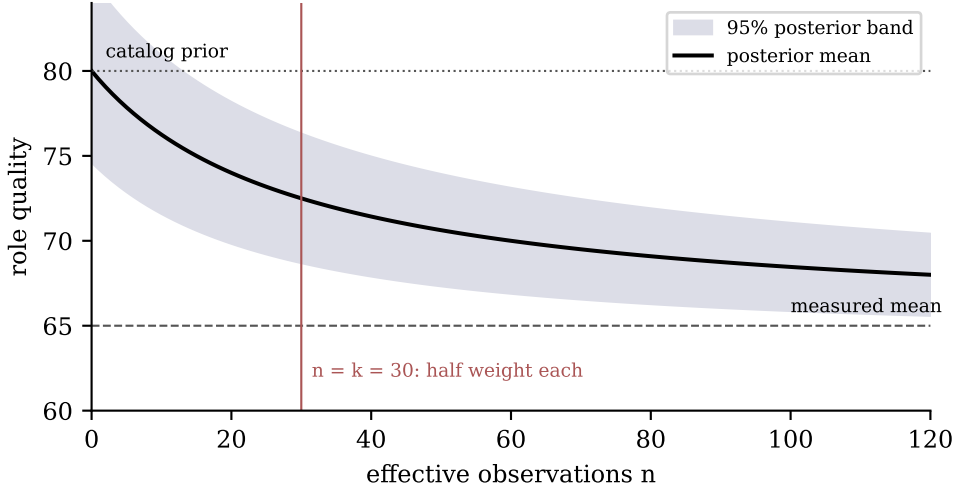


Figure 3: Shrinkage of one role quality from the catalog prior (80) toward the measured mean (65), $k = 30$, $\sigma = 15$. The band is what the Thompson draw of Section 7 samples from.

Algorithm 1 One pick under *learn*

Require: shortlists S_r on the posterior means, budget word $\Rightarrow H$, task size bucket $\Rightarrow$ expected tokens T and minutes $t(\kappa)$

- 1: **for** each role r and $m \in S_r$ **do**
- 2: $\tilde{q}_{rm} \sim \mathcal{N}(\mu'_{rm}, \sigma_{rm}^2)$ ▷ one draw per run; never redrawn during the run
- 3: **end for**
- 4: **for** each $\kappa \in \mathcal{K}$ **do**
- 5: $\tilde{E}(\kappa) \leftarrow T B(\kappa) + \lambda t(\kappa) + H (1 - \sum_r v_r \tilde{q}_{r\kappa(r)}/100)$
- 6: **end for**
- 7: **return** $\arg \min_{\kappa} \tilde{E}(\kappa)$ ▷ ties: fewer imputed indexes, then id order

The structure that matters is the feedback. A run of crew κ is judged role by role, so it yields one observation for each of its n (role, model) pairs, and the posterior of every crew that shares a pair with κ moves. This is a combinatorial bandit with semi-bandit feedback [4, 5]: the unknowns are the $\sum_r |S_r|$ role qualities, not the $|\mathcal{K}|$ crew values, and the regret of Thompson sampling scales with the former [5]. Section 8 measures the difference on our runs. For Gaussian posteriors the Bayesian regret of Thompson sampling is $O(\sqrt{dT \log T})$ in the dimension d of the unknowns over T runs [3].

The learned front. Under *learn* the front of Section 3 is recomputed on the posterior: the bill $B(\kappa)$ is read at the run’s expected tokens, and the quality is the posterior mean of $P_c(\kappa)$, so the front is the pareto set in (expected dollars per task, expected probability of a clean delivery) with an interval on each point from (15). The budget word’s H is a line of slope $-1/H$ in that plane and the pick is where the line first touches the front; Algorithm 1 is the same pick on one draw of the front rather than on its mean, which is what makes it explore. The pool command prints the mean front with its intervals, and the catalog front beside it, so that a user can see where measurement moved a pick and by how much.

When is it settled. The same posterior answers whether learning is done: the probability π^* that the posterior-mean crew is the best, estimated by repeating the draw of Algorithm 1 a few hundred times. The pool command reports π^* ; a value near one means further exploration is nearly free, a value near $1/|\mathcal{F}|$ means the ranking is still open. Costs enter as observed dollars per run and are not sampled; their noise is small next to the quality’s.

Remark 1 (A new model). A model with no rows has $n = 0$ and sits at its catalog prior with the prior variance σ^2/k^* . If that puts it near the front it is drawn into crews soon, judged, and confirmed or pushed down by its first rows. Nothing is done by hand when the catalog changes.

8 Experiments on our measured runs

Ground truth is the table of Figure 1: 56 crews with P_c , dollars and minutes fitted by a ridge model on 72 scored runs of our own over nine tasks. Each replicate draws a catalog prior per arm as the truth plus $\mathcal{N}(0, 0.15^2)$ noise, roughly the disagreement between indexes and measurements in Figure 6, holds it with $k = 30$, and runs N tasks; a strategy picks, observes the outcome and pays (8) with $H = \$10$. Regret is the excess over the oracle crew, averaged over 200 replicates.

Crew-level learners. Table 2 and Figure 4 compare, on all 23 chat-door crews of the table, a fixed table row (the shipped balanced crew), the catalog prior alone, ϵ -greedy and Thompson, all with one arm per crew. After 400 runs Thompson is \$0.33 per task under the fixed table, a third of the table’s regret, and still falling; ϵ -greedy is \$0.12 behind it. Thompson pays an exploration tax in its first forty runs, when it is no better than the table; $k = 30$ is what bounds it. ϵ -greedy pays more, because a tenth of its runs go to a random crew among 23, and its rate neither grows when a new model appears nor shrinks when the ranking is settled.

N	10	20	40	70	100	150	200	300	400
table	1.01	0.95	0.95	0.93	0.95	0.93	0.94	0.96	0.97
catalog	0.99	0.94	0.94	0.91	0.92	0.88	0.88	0.90	0.91
greedy	1.20	1.10	1.01	0.99	0.94	0.90	0.86	0.80	0.76
thompson	0.97	0.97	0.93	0.87	0.88	0.84	0.79	0.71	0.64

Table 2: Regret against the oracle, dollars per task after N runs, one arm per crew.

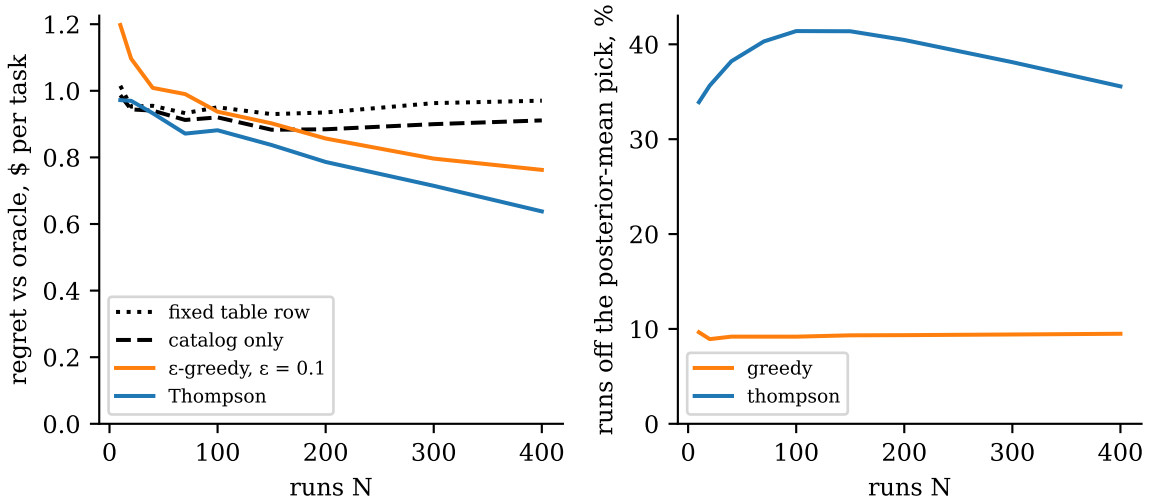


Figure 4: Left: regret per task. Right: share of runs on a crew other than the current posterior-mean best. Thompson explores heavily while many crews sit within cents of each other and less as the posteriors separate; ϵ -greedy explores at its fixed rate.

Role-level feedback. Figure 5 and Table 3 isolate the structure of Section 7. The ground truth is three-role: a role quality per (role, model) read off the measured rows (a worker’s delivered rate, a checker’s and a planner’s clean rate), combined by the linear rule of Assumption 1 over every crew the vendor rule admits, with dollars additive over roles. Three learners run Thompson sampling with the same prior noise and the same H and differ only in what they see after a run: one arm per crew fed the run’s clean-or-defect bit; one arm per crew fed the mean of the judge’s role scores; one posterior per (role, model) fed each role’s own score, which is what the judge of Section 5.1 provides. The last is the semi-bandit. Richer feedback per run helps: the crew-level learner fed the mean score sits at \$0.31 after four hundred runs against \$0.54 for the one fed only the clean bit. The structure helps again as much: the role-level learner, fed the same scores as the second, sits at \$0.15, half of it and a third of the first, because every run updates the crews that share a model with the one that ran, and there are sixteen unknowns behind 138 crews.

N	10	20	40	70	100	150	200	300	400
crew, clean bit	0.47	0.50	0.50	0.51	0.53	0.54	0.55	0.55	0.54
crew, mean score	0.29	0.43	0.39	0.38	0.36	0.35	0.34	0.34	0.31
role, semi-bandit	0.36	0.35	0.31	0.27	0.26	0.22	0.20	0.17	0.15

Table 3: Regret against the oracle, dollars per task after N runs, on the three-role truth; same prior noise and H for all three learners.

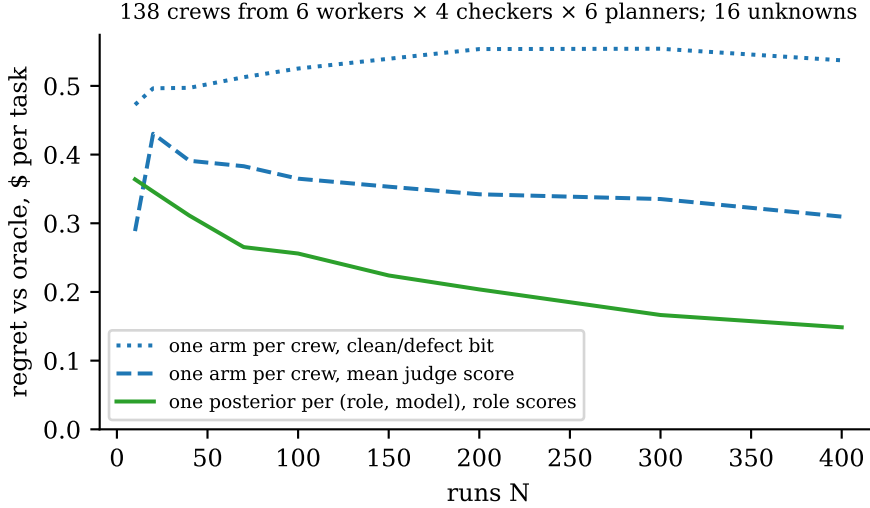


Figure 5: Thompson sampling under three kinds of feedback on the same three-role truth. Dotted and dashed: one arm per crew, fed the clean bit and the mean role score respectively. Solid: one posterior per (role, model) fed the role scores, the combinatorial semi-bandit of Section 7.

The truth in these experiments is our own fitted table from one team’s tasks, so the size of the gain is the shape of the effect and not a promise for every install; the regret scale is set by how close the crews sit, which the front of Figure 1 shows and no bound here uses. The script `figures.py` beside this document reproduces every figure and table from the two CSV files it reads and the constants written at its head.

9 System architecture

The statistics of Sections 5–7 fix what the system computes; this section fixes where the bytes live, so that the privacy claims can be checked against the deployment rather than taken on trust. Figure 7 shows the four components. The design rule is that the relay holds nothing worth stealing and nothing that cannot be rebuilt.

The binary. Everything that touches a task runs inside the install: the catalog read, the posterior, the draw, the run, the judge call, the own sheet and the outbox. The judge is an ordinary model call through the install’s own provider credentials; the record it reads never leaves the machines the install already trusts with its work. The outbox is a newline-delimited file under the profile directory; `codeaf pool status` prints its pending rows, and a user who wants to see exactly what would be sent can read the file. The loader reads the fresher of the cached index and the embedded seed index, by generated day, once at start-up; a refresh runs in the background and takes effect at the next start, so a running task’s picks never move under it.

The relay. The reference relay is a single edge function with an object store behind it, deployable in minutes on a commodity edge platform (`wrangler deploy` from `relay/` in the repository; the runbook in that directory is ten steps). It exposes two routes. `POST /v1/rows` accepts a batch of rows, validates the schema, folds each row into the sender’s triple for its cell and day, and discards the row: the store holds the sheet of Definition 2 and nothing else, one triple per (install, cell, day), so its size is bounded by installs times cells times days and not by traffic, and a row that has been folded cannot be recovered from it. `GET /index.json` serves the last published index and its

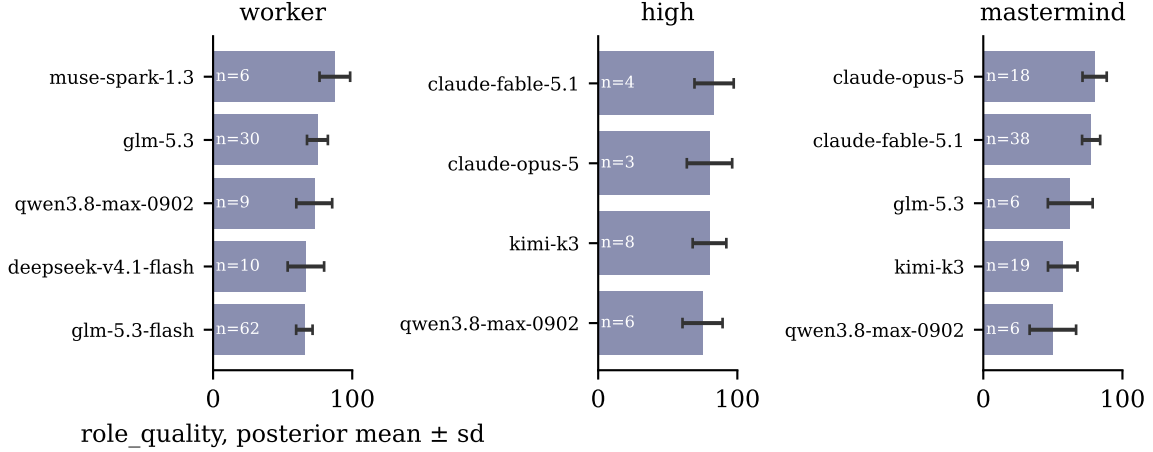


Figure 6: The seed index shipped in the binary: 14 cells from 120 scored runs, $\text{role_quality} = 100 \cdot P(\text{no major defect})$ under a $\text{Beta}(1,1)$ posterior. The spread between models a catalog rates alike is why the pool exists.

detached signature. An hourly scheduled job merges the sheet, runs the judge-effect, robust-aggregation and drift estimates of Section 5, writes the index, signs it with an Ed25519 key held as a platform secret, and mirrors the pair of files to a public git branch so that they are browsable, diffable and forkable. The function keeps no request logs and stores no network address; the install nonce is the only identity in the store, and rate limiting is by nonce (a cap on rows per day) rather than by address.

Keys and verification. The signing key’s public half is compiled into the binary as a list, so a key can be rotated by shipping a binary that carries both; the private half exists only in the platform secret store. Verification is client-side: the binary checks the signature before it reads a single cell, refuses a version lower than the one it holds (a rollback is caught by the version; the join of Section 5.2 makes a replayed old publication harmless), and on any failure keeps the cached copy beside an error. Because verification is local, the index can be served from anywhere: the mirror, a static file host, or a directory on disk, which is what the `file://` address is for.

Trust and failure. What a compromised relay learns is the sheet: counts, sums and sums of squares of scores per (install nonce, role, model, judge, day, size, door). It learns no task, no code, no prompt, no user, and no address. What a compromised relay can do is publish a bad index, which the signature stops unless the key is also taken; a taken key is retired by shipping the next key, and the min-installs and robust-aggregation rules bound what a flood of fabricated nonces can move in the meantime. What an unreachable relay costs is nothing at all: the cache, then the seed, answer every read, and the outbox waits. Any install can run the same function against its own store and point its `CODEAF_MODEL_POOL_URL` and `_SUBMIT_URL` at it; two relays that receive the same publications produce the same sheet (Proposition 2), so a private relay and the public one can be federated later by a one-way copy of publications with no reconciliation step.

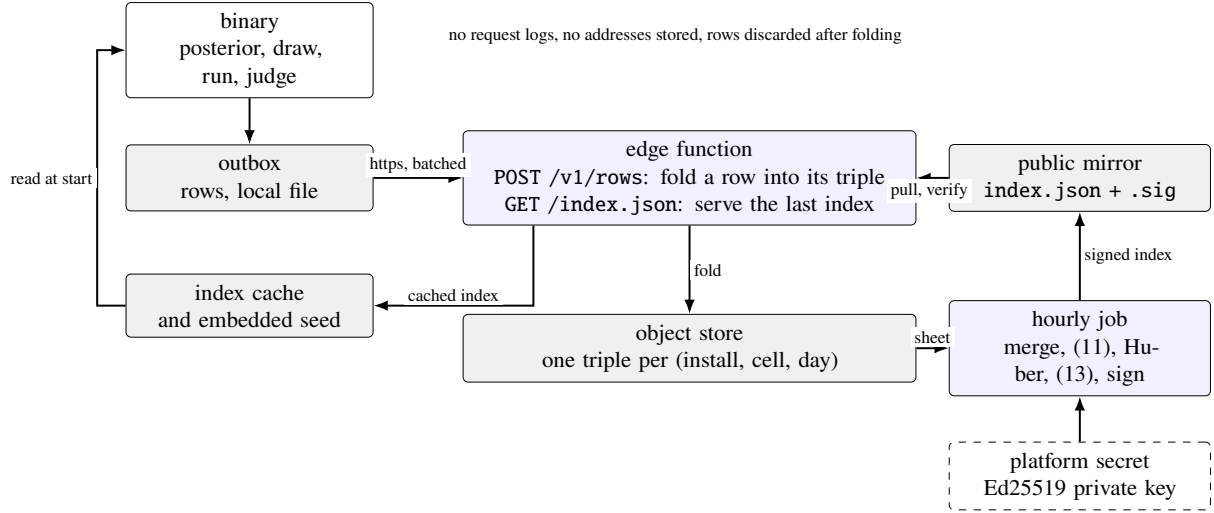


Figure 7: Deployment. The relay is one edge function, one object store holding only the sheet, and one scheduled signing job; the install verifies every index locally and never blocks on the network. Any of the three network components can be replaced by a static file, a directory, or a self-hosted copy.

10 What the user controls

setting	values	meaning
crew	frugal balanced max custom	how much to spend: H and the knob k
picked from	table catalog learn	the rows we measured; recomputed from today’s catalog; catalog plus pool plus own sheet
model_pool	on read off	read the index and send rows (default); read only; neither
explore	on off	Thompson draw (default) or the posterior mean; <i>learn</i> only; specified, not on this head

With the pool set to *read*, the own sheet still accrues and enters (15); with it *off*, only the catalog prior remains and the pick is the catalog’s. The index is a JSON document signed with an Ed25519 key whose public half is in the binary; `codeaf pool verify` checks it, `pool show` prints its summary and each address with its source, `pool status` counts the rows waiting in the outbox and says whether the relay answered. The binary carries a seed index of our own scored runs (Figure 6) so that *learn* has numbers on day one. The relay’s code is in this repository; its addresses are settings, so an install can read from and send to a relay of its own, or to none.

11 Status

On the head this document ships with: the catalog crew of Section 3; the shrinkage blend of the Corollary with $k = 30$; the `picked from` row and its ladder; the index document, its reader and signed pull with the built-in key; the outbox, the install nonce and the push; the judge, its deterministic choice, and the hook that calls it when a chat task lands; the own sheet, whose cells enter the prior beside the index’s by observation count; the settings; the seed index; the relay of Section 9 with its store keyed by install and day, the join, the judge-effect fit and the Huber aggregation; the mirror workflow. Specified here and next to land: the drift estimate (13); the hierarchical cap (16) with the moment estimators (17)–(20) in place of the count fold; the Thompson draw with role-level updates, the `explore` setting and the settled probability π^* ; the pair term of Section 6.1; the headless doors’ hook. The referee’s report on an earlier draft, and what changed in answer to it, are beside this file.

References

- [1] W. R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika* 25:285–294, 1933.
- [2] D. Russo, B. Van Roy, A. Kazerouni, I. Osband, Z. Wen. A tutorial on Thompson sampling. *Foundations and Trends in Machine Learning* 11(1):1–96, 2018.

- [3] D. Russo, B. Van Roy. Learning to optimize via posterior sampling. *Mathematics of Operations Research* 39(4):1221–1243, 2014.
- [4] W. Chen, Y. Wang, Y. Yuan. Combinatorial multi-armed bandit: general framework and applications. *ICML*, 2013.
- [5] Z. Wen, B. Kveton, A. Ashkan. Efficient learning in large-scale combinatorial semi-bandits. *ICML*, 2015.
- [6] B. Efron, C. Morris. Data analysis using Stein’s estimator and its generalizations. *Journal of the American Statistical Association* 70(350):311–319, 1975.
- [7] M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski. Conflict-free replicated data types. *SSS*, 2011.
- [8] P. J. Huber. Robust estimation of a location parameter. *Annals of Mathematical Statistics* 35(1):73–101, 1964.
- [9] A. C. Harvey. *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge University Press, 1989.